

Unit -2

Introduction to Android Programming [4 HRS]

- Android Platform,
- History of Android,
- Environment Setup,
- Creating an android project,
- Laying out the user interface (The view hierarchy, widget attributes, creating string resources, previewing the layout),
- Creating a new class,
- Setting up the project,
- Running on the Emulator.

Android Platform

Android is an open-source mobile operating system (OS) developed by Google. It is based on the Linux kernel and is designed primarily for touchscreen mobile devices such as smartphones and tablets.

- Android is open source and highly customizable allows manufacturers to modify it for their specific hardware needs and software preferences.
- Android apps are primarily written in Java and Kotlin programming languages, using the **Android Software Development Kit (SDK)**. It provides access to a large number of apps through the Google Play Store and other app stores.
- Android has a **large developer community** and extensive documentation
- Android supports a wide range of hardware features, such as Bluetooth, NFC and GPS.
- Android has several **built-in security** features, such as sandboxing, permission-based app access, and regular security updates.
- Android is continually evolving, with new versions released regularly that add new features and improvements.

History of Android

Year	Event
2003	Android Inc. founded by Andy Rubin, Rich Miner, Nick Sears, and Chris White
2005	Google acquires Android Inc.
2007	Open Handset Alliance (OHA) formed, consisting of Google, HTC, Samsung, and others
2008	Android 1.0 (no official codename) released on September 23
2009	Android 1.5 Cupcake released on April 27
2009	Android 1.6 Donut released on September 15
2010	Android 2.0/2.1 Éclair released on October 26
2011	Android 2.2/2.2.3 Froyo released on May 20
2011	Android 2.3/2.3.7 Gingerbread released on December 6
2011	Android 3.0/3.1/3.2 Honeycomb released on February 22 (tablet-only)
2011	Android 4.0/4.0.4 Ice Cream Sandwich released on October 18

- Honeycomb was the only tablet-specific version of Android
- Ice Cream Sandwich combined the tablet and phone versions of the OS, but completely overhauled (Rebuild) the look and feel of the system.

History of Android

Year	Event
2012	Android 4.1/4.2/4.3 Jelly Bean released on July 9
2013	Android 4.4 KitKat released on October 31
2014	Android 5.0/5.1 Lollipop released on November 12
2015	Android 6.0 Marshmallow released on October 5
2016	Android 7.0/7.1 Nougat released on August 22
2017	Android 8.0/8.1 Oreo released on August 21
2018	Android 9 Pie released on August 6
2019	Android 10 released on September 3
2020	Android 11 released on September 8
2021	Android 12 released on October 4
2022	Android 13.0 released August 15

Android Architecture or Stacked

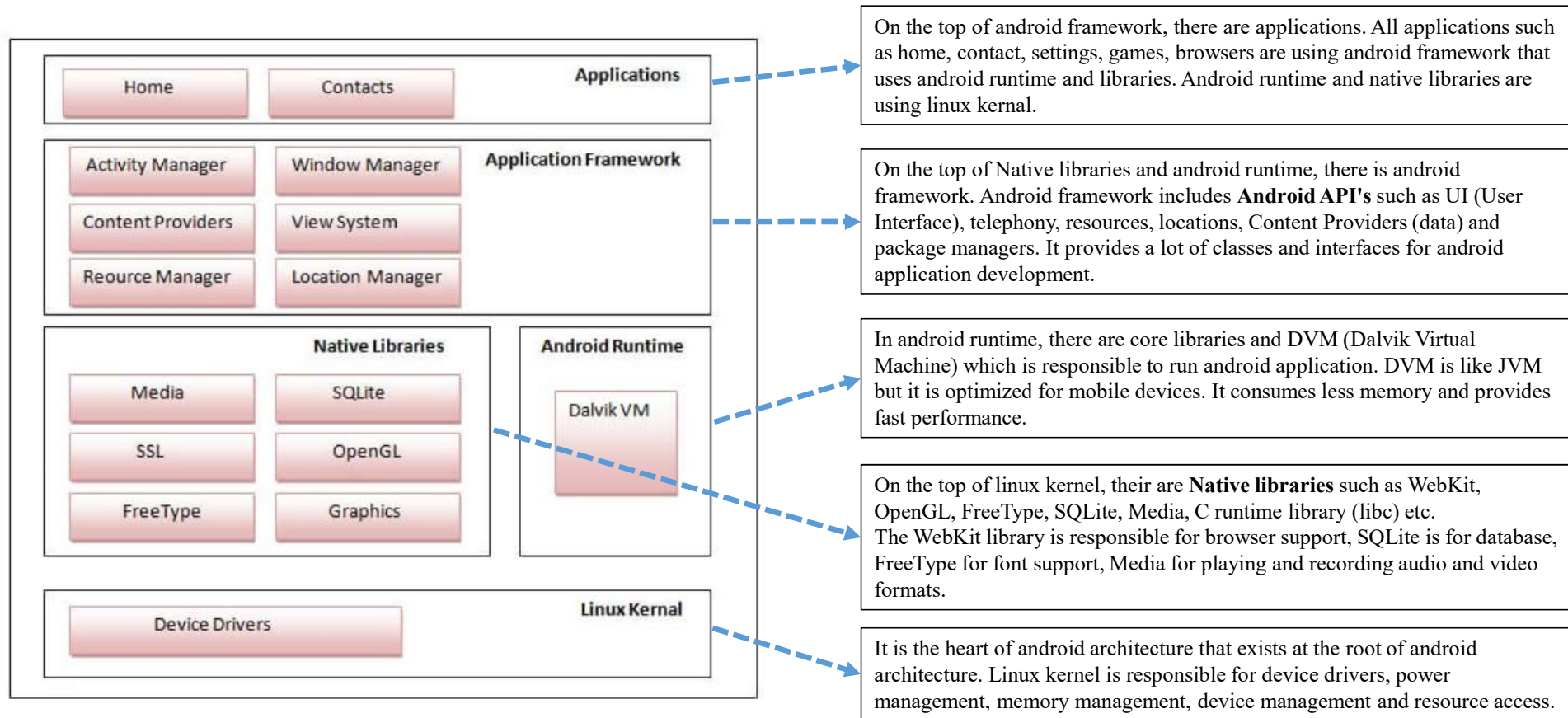
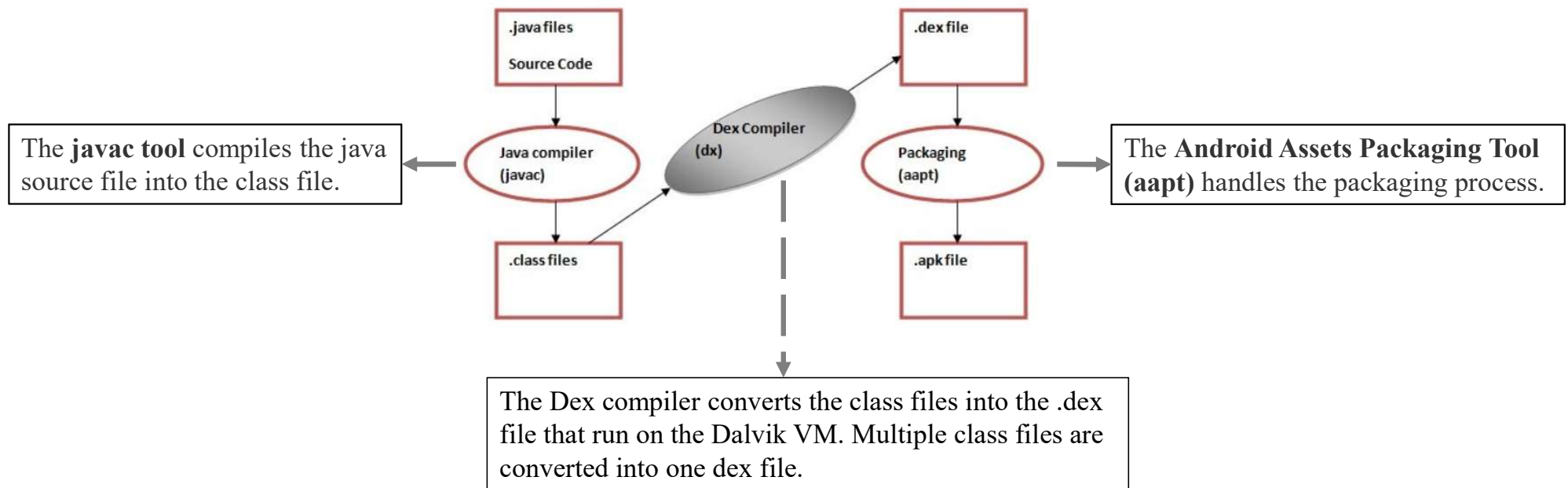


Image Source: <https://static.javatpoint.com/images/androidimages/softwarestack.jpg>

Dalvik Virtual Machine (DVM)

- **DVM** is an android virtual machine optimized for mobile devices. It optimizes the virtual machine for memory, battery life and performance. [DVM is a optimized JVM for low-powered handheld devices by utilizing its high performance and excellent memory management features]
- Dalvik is a name of a town in Iceland. The Dalvik VM was written by Dan Bornstein.



Core Building Blocks / Fundamentals of Android

1. **Activity:** An activity is a class that represents a single screen. It is like a Frame in AWT (**Abstract Window Toolkit**).
2. **View:** A view is the UI element such as button, label, text field etc. Anything that you see is a view.
3. **Intent:** Intent is used to invoke components. It is mainly used to: Start the service, Launch an activity, Display a web page, Display a list of contacts, Broadcast a message, Dial a phone call etc.

For example, you may write the following code to view the webpage.

```
Intent intent=new Intent(Intent.ACTION_VIEW);  
intent.setData(Uri.parse("http://www.javatpoint.com"));  
startActivity(intent);
```

4. **Service:** Service is a background process that can run for a long time. There are two types of services local and remote. Local service is accessed from within the application whereas remote service is accessed remotely from other applications running on the same device.
5. **Content Provider:** Content Providers are used to share data between the applications.
6. **Fragment:** Fragments are like parts of activity. An activity can display one or more fragments on the screen at the same time.
7. **AndroidManifest.xml:** It contains information's about activities, content providers, permissions etc. It is like the web.xml file in Java EE.
8. **Android Virtual Device (AVD):** It is used to test the android application without the need for mobile or tablet etc. It can be created in different configurations to emulate different types of real devices.



Environment Setup

1. **Follow system requirements**
2. **Setup your java environment [JDK and JRE]**
3. **Setup your Android Environment:** To develop Android apps, you must download **Android Studio** [official IDE (Integrated Development Environment) developed by the JetBrains community which is freely provided by Google for android app development] and the Android SDK.

Complete the following steps to install Android Studio and the Android SDK:

- Go to the Download the **Android SDK** and SDK Tools page.
- Select and download the **Android Studio** package for your operating system (one of Windows, MacOS, or Linux).
- Unzip and install the bundle. It includes essential components and the Studio IDE.
- Launch Android Studio.
- Download the latest **SDK tools** and **SDK platforms** using the **Android SDK Manager**.
- To open the **SDK Manager**, click *Tools > Android > SDK Manager*.
- Install updated packages if needed.

Creating an android project

Creating the new android project

- 1) Select Start a new Android Studio project
- 2) Provide the following information: Application name, Company domain, Project location and Package name of application and click next.
- 3) Select the API level of application and click next.
- 4) Select the Activity type (Empty Activity).
- 5) Provide the Activity Name and click finish.

After finishing the Activity configuration, Android Studio auto generates the activity class and other required configuration files.

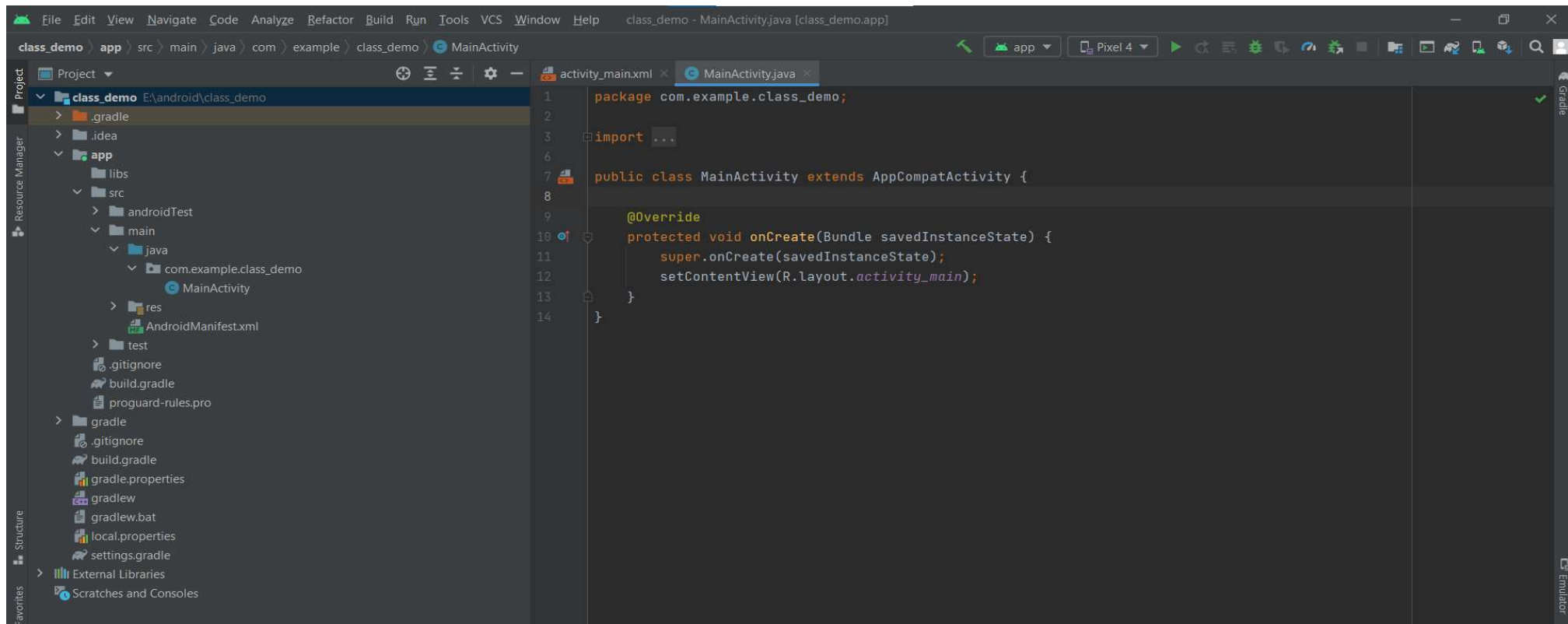
Write the message

Edit : *File: activity_main.xml [auto-generated file by Android Studio]*

Run the android application

To run the android application, click the run icon on the toolbar or simply press Shift + F10.

Android Project folder Structure [Android Studio]



Android Project folder Structure [Android Studio]

The android project contains different types of app modules, source code files, and resource files.

1. **Manifests Folder:** Manifests folder contains AndroidManifest.xml for creating our android application. This file contains information about our application such as the Android version, metadata, states package for Java or Kotlin file, and other application components. It acts as an intermediary between android OS and our application.
2. **Java Folder:** The Java folder contains all the Java or Kotlin source code (.java) files that we create during the app development, including other Test files.
3. **res (Resources) Folder:** The resource folder is the most important folder because it contains all the non-code sources like images, XML layouts, and UI strings for our android application.
 - a) *Drawable Folder:* It contains the different types of images used for the development of the application. We need to add all the images in a drawable folder for the application development.

Android Project folder Structure [Android Studio]

- b) Layout Folder:* The layout folder contains all XML layout files which we used to define the user interface of our application. It contains the activity_main.xml file.
- c) Mipmap Folder:* This folder contains launcher.xml files to define icons that are used to show on the home screen. It contains different density types of icons depending upon the size of the device such as hdpi, mdpi, xdpi.
- d) Values Folder:* Values folder contains a number of XML files like strings, dimensions, colors, and style definitions. One of the most important files is the strings.xml file which contains the resources.

4. Gradle Scripts: Gradle means automated build system and it contains a number of files that are used to define a build configuration that can be applied to all modules in our application. In build.gradle (Project) there are buildscripts and in build.gradle (Module) plugins and implementations are used to build configurations that can be applied to all our application modules.

Next Class

1. Laying out the user interface (The view hierarchy, widget attributes, creating string resources, previewing the layout)
2. Creating a new class
3. Setting up the project
4. Running on the Emulator

View hierarchy

- In Android, the view hierarchy is a tree-like structure that represents the user interface of an app. Each node in the view hierarchy is a View object or one of its subclasses. The root node is typically a ViewGroup object, such as a LinearLayout or RelativeLayout, which contains one or more child views.
- The view hierarchy defines the layout of an app's user interface, and it determines how the views are rendered on the screen. Here's an example of a simple view hierarchy:

LinearLayout

- **TextView**
- **EditText**
- **Button**

- In this example, the root node is a LinearLayout, which contains three child views: a TextView, an EditText, and a Button. The LinearLayout arranges these child views vertically, one after the other.

View hierarchy [Contd...]

Here's an explanation of each element in this view hierarchy:

- **LinearLayout:** A ViewGroup that arranges child views in a linear fashion, either horizontally or vertically.
- **TextView:** A View that displays text on the screen.
- **EditText:** A View that allows the user to enter text.
- **Button:** A View that represents a button that the user can click.

Each view in the view hierarchy has its own set of attributes that can be used to control its appearance and behavior. For example, the TextView has attributes such as text, textSize, textColor, etc. The EditText has attributes such as hint, inputType, etc. The Button has attributes such as text, onClick, etc.

To create a view hierarchy in Android, you can use the layout editor in Android Studio. You can drag and drop views from the palette onto the design surface, and you can use layout containers such as LinearLayout, RelativeLayout, etc. to arrange the views in a particular layout. You can also set attributes for each view using the Properties window.

Widgets in Android

Widgets are the basic building blocks of an app's user interface. Widgets are visual elements that are used to display information, receive user input, and perform actions. Here are some commonly used widgets in Android:

- **TextView:** A widget used to display text on the screen.
- **EditText:** A widget used to receive text input from the user.
- **Button:** A widget used to trigger an action when clicked.
- **ImageView:** A widget used to display images on the screen.
- **ProgressBar:** A widget used to show the progress of an operation.
- **CheckBox:** A widget used to allow the user to select one or more items from a list.

Widgets in Android

[Contd...]

- **RadioButton:** A widget used to allow the user to select one option from a set of mutually exclusive options.
- **ToggleButton:** A widget used to switch between two states, such as on and off.
- **Spinner:** A widget used to allow the user to select an item from a dropdown list.
- **SeekBar:** A widget used to allow the user to select a value from a range of values by dragging a slider.
- **DatePicker:** A widget used to allow the user to select a date from a calendar.
- **TimePicker:** A widget used to allow the user to select a time from a clock.

These are just a few examples of the many widgets available in Android. Each widget has its own set of attributes and methods that can be used to customize its appearance and behavior. By combining different widgets and arranging them in layouts, you can create complex user interfaces for your Android app.

Widget Attributes

In Android, Widget attributes are used to customize the appearance and behavior of views. Each view has its own set of attributes that can be used to control its appearance, layout, and interaction with the user. Here are some commonly used widget attributes in Android:

- **android:id:** This attribute is used to assign a unique ID to a view. The ID is used to reference the view in code, for example to set its text or visibility.
- **android:text:** This attribute is used to set the text of a view, such as a TextView or Button.
- **android:layout_width and android:layout_height:** These attributes are used to set the width and height of a view, either to a fixed size or to match the size of its parent container.
- **android:layout_margin:** This attribute is used to set the margin around a view, which determines the spacing between the view and its parent container or other views.

Widget Attributes [Contd...]

- **android:layout_gravity:** This attribute is used to set the gravity of a view, which determines its position within its parent container.
- **android:background:** This attribute is used to set the background color or image of a view.
- **android:textSize:** This attribute is used to set the size of the text in a view.
- **android:padding:** This attribute is used to set the padding around the content of a view, which determines the spacing between the content and the view's border.
- **android:onClick:** This attribute is used to set the method to be called when the view is clicked.
- **android:visibility:** This attribute is used to set the visibility of a view, either to visible, invisible, or gone.

These are just a few examples of the many widget attributes available in Android. Each view has its own set of attributes, and different layouts and view groups may have additional attributes. When designing an Android user interface, it is important to choose the appropriate attributes to achieve the desired appearance and behavior of the views.

Creating string resources

In Android, string resources are used to store text that will be displayed in the user interface. By using string resources, you can easily localize your app and provide translations for different languages. Here are the steps to create a string resource in Android:

Open the strings.xml file: This file is located in the res/values folder of your Android project.

Define a new string resource: Inside the strings.xml file, define a new string resource using the following syntax:

```
<string name="string_name">string_value</string>
```

Replace string_name with a unique name for your string resource, and string_value with the text you want to display.

Creating string resources [Contd...]

Use the string resource in your layout or code: To use the string resource, reference it using the `@string/` syntax in your layout XML files or Java code. For example:

```
<TextView  
    android:id="@+id/my_text_view"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="@string/hello_world" />
```

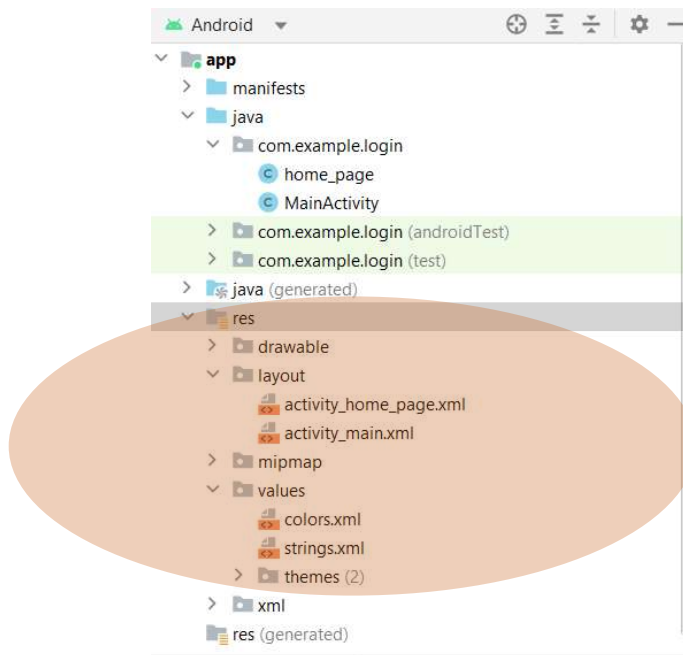
In this example, the string resource named `hello_world` is used to set the text of a `TextView`.

Create additional string resources: Repeat steps 2 and 3 for each string you want to use in your app.

By using string resources, you can easily update the text displayed in your app without having to modify your layout files or code. This is especially useful when localizing your app for different languages, as you can provide translations for each string resource.

Previewing the layout

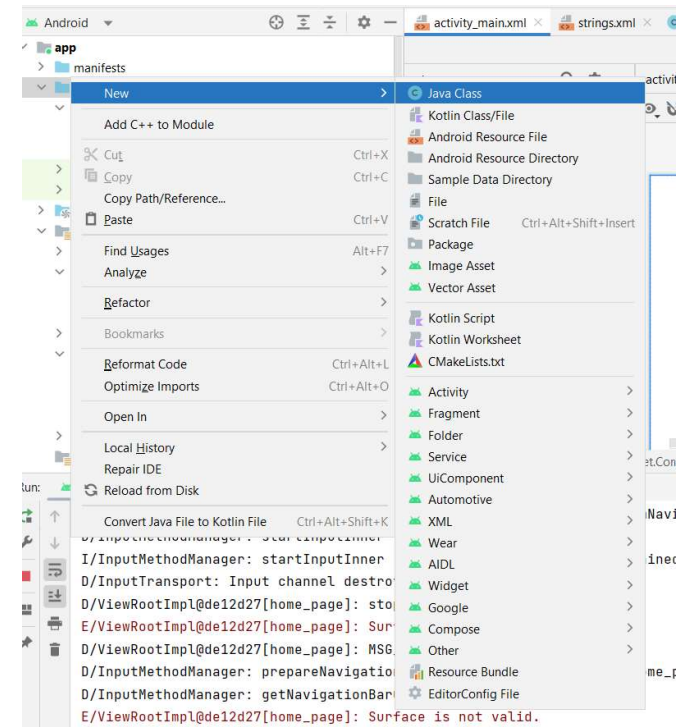
In Android Studio, you can preview the layout of your app in real-time using the Layout Editor. The Layout Editor is a visual editor that allows you to drag-and-drop widgets onto the screen, and then customize their properties using the Properties panel.



Creating a new class

In Android Studio, you can create a new class using the following steps:

1. Open your project in Android Studio.
2. Right-click on the package where you want to create the new class.
3. Select **New -> Java Class** from the context menu.
4. In the New Java Class dialog, enter the name of the class you want to create.
5. Choose the superclass for your new class. If your class doesn't need a superclass, you can select "Class" as the superclass.
6. Choose the package where you want to create the new class.
7. Click on the "Finish" button to create the new class.



Creating a new class **[Contd...]**

- Once you've created the new class, you can start adding code to it. By convention, class names in Java start with an uppercase letter and use camel case, where each word in the name is capitalized except for the first word. For example, if you were creating a class to represent a user in your app, you might name it "User".
- It's important to note that if you're creating a new activity in your Android app, you should extend the "Activity" class. Similarly, if you're creating a new fragment, you should extend the "Fragment" class. These classes provide the necessary functionality to create UI elements and handle user input.

Dragging Views to Left, Right, Top and Bottom of the Android Layout

- When designing an Android layout, dragging views to the left, right, top, and bottom helps in positioning views relative to each other within a container or the parent layout. This is known as using **layout constraints**.
- Layout constraints are used to specify **the position of a view relative to its container** or other views in the layout. For example, if you want to position a button at the bottom right corner of a parent layout, you would drag the button towards the bottom and the right edge of the parent layout until it snaps to the edges. This will create layout constraints that ensure the button remains anchored to the bottom and right edges of the parent layout.
- Using layout constraints is a more flexible and powerful way of positioning views compared to using absolute positions. With layout constraints, you can specify rules such as aligning views to the left or right of each other, centering views horizontally or vertically, setting minimum and maximum dimensions, and more.
- Layout constraints can be created using different layout containers such as `LinearLayout`, `RelativeLayout`, `ConstraintLayout`, etc. In `ConstraintLayout`, layout constraints are created by dragging views to the edges of the container or other views and setting anchor points using guidelines.
- By using layout constraints, you can create responsive and flexible layouts that can adapt to different screen sizes and orientations.

“ Have a Good Day ”

Thank You